

Proving the Program You Run

Mutable borrows inside a dependent type theory, by observation and closing off

Abstract. Verified systems code today is written twice: once as the efficient imperative program that runs, and once as a pure model that the proofs are about, joined by a refinement proof or a translation. We present Ochr, a core calculus in which the two are the same text. Ochr combines Rust-style mutable borrows with a Lean-style dependent type theory, and its definitional equality unfolds in-place code exactly as Lean unfolds pure code. Three ideas make this work. First, an *observational* equality `Id A t u` compares two computations by what they return and what they leave behind in the places they may write; it is not a new primitive but computes to ordinary equality between observations. Second, a call whose body is stuck on an unknown value is *closed off*: its result, and the final content of every place it borrowed, become *sealed programs*, closed source programs that own their state. Sealed programs are the backward functions of Aeneas, but written in the source language, so the programmer never sees a second language. Third, induction hypotheses are typed in the caller’s environment, where the borrow structure around the recursive argument performs the congruence step that a pure proof would write by hand. Proofs about in-place functions become plain structural recursion, frequently shorter than the corresponding proofs about pure functions. We give the calculus, its worked examples (including a function that returns a borrow into its argument), a translation into the calculus of inductive constructions that establishes consistency, and an executable Lean implementation of the checker.

1 Introduction

Here is an addition on Peano numerals that allocates nothing. It walks down the successors of `*x` until it reaches the final `Z` and moves `y` into that spot:

```
AddM : Π(x : &Nat) (y : Nat). Unit
AddM x y := match *x { Z => *x := y | S p => AddM &p y }
```

`&Nat` is a mutable borrow in the sense of Rust: a unique, temporary right to read and write a `Nat` owned by someone else. In the successor case the pattern variable `p` names the predecessor field in place, and `&p` reborrows it for the recursive call. Here is a theorem about `AddM`, stated and proved in the same language:

```
AddMZero : Π(x : &Nat). Id Unit (AddM x 0) ()
AddMZero x := match *x { Z => refl | S p => AddMZero &p }
```

The statement says that running `AddM x 0` is indistinguishable from doing nothing: both return `()`, and both leave the number behind `x` as it was. The proof is structural recursion and nothing else. There is no pure model of `AddM`, no refinement relation, no loop invariant, no separation-logic assertion and no translation into another language. The checker accepts the recursive call as a proof of the successor case by running both sides of the equation on a symbolic input and comparing what they leave behind.

This paper is about the type theory that makes that possible.

1.1 The two-language problem

Verified software is usually built in one of two ways. In the first, everything is written in a pure proof assistant such as Lean or Coq, and performance comes from the compiler’s ability to turn functional updates into in-place ones where it can prove the old value is dead. In the second, the efficient program is written in an imperative language, and a separate pure specification is related to it by a refinement proof: Verus [11] and Creusot [7] check Rust against specifications discharged by SMT, VeriFast checks C against separation-logic contracts, and Aeneas [9] translates safe Rust into a pure Lean program about which the user then proves theorems.

Both routes keep two artifacts. In the second route this is visible: there is a program and there is a specification, and the proof is about their relationship. In the first route it is hidden but present: the program the proof is about is not the program that runs, and the gap is bridged by an optimisation the programmer cannot state or rely on.

Aeneas comes closest to closing the gap. Its insight is that Rust’s ownership discipline makes mutation equivalent to state passing: a function taking $x : \&\text{mut } T$ is a pure function returning, besides its result, a *backward function* that computes the final value of $*x$. Aeneas performs this translation once, eagerly, for a whole program, and the user proves theorems about the translated Lean code, where the backward functions appear as ordinary definitions. The user therefore works in two languages at once, and must understand how one maps onto the other. We show that this translation can be performed **inside the type checker, lazily, as definitional equality, and in the source language itself.**

1.2 Three ideas

Observational equality. The type $\text{Id } A \ t \ u$ compares two computations t and u of result type A , run from the current state. It holds when they return equal values and leave equal contents in every place they may write. Id is not a new primitive with its own introduction and elimination rules. At a given environment it computes to an ordinary propositional equality between two *observations*, the tuples of result and final contents, and that equality computes further by the structure of the data in the manner of observational type theory [4]. The places a computation may write are read off its syntax, so they are an output of the computation like its result. This is the sense in which Ochr has no pure/impure divide: any program may appear in a type, and what it does to its environment is part of what the type talks about.

Closing off. Definitional equality in Ochr is normalisation by a deterministic symbolic machine: two terms are equal when the machine sends them to the same normal form. On symbolic inputs the machine gets stuck, and a dependent type theory needs a name, a *neutral term*, for every stuck computation. A stuck pure computation names itself: Lean’s $\text{Nat.add } n \ 0$ is its own normal form. A stuck effectful call does not, because part of what it produces lives in the places it borrowed. We *close off* such a call: its result and the final content of each borrowed place are replaced by *sealed programs*, closed source programs that own their state. For AddM on an unknown σ , the borrowed place receives $[\text{let } c = \sigma; \text{AddM } \&c \ 0; c]$, which is the pure wrapper a programmer would write anyway. When a later case split refines σ , the sealed program runs again. Sealed programs are exactly Aeneas’s backward functions, but they are written in the source language and generated on demand, so the programmer never writes or reads a second language. A function that returns a borrow is closed off into a sealed program with a hole awaiting the final value of the returned borrow; this is Aeneas’s region abstraction, in the same form.

The environment does the congruence. When the checker types the recursive call $\text{AddMZero } \&p$, it evaluates the callee’s statement in the caller’s environment, where $\&p$ points into the predecessor field of the caller’s number. The observation therefore sees the successor that surrounds the recursive call, and the induction hypothesis arrives already wrapped in S : it is literally the goal. The step $\text{cong } S \ ih$ that a pure proof writes by hand has been performed by the borrow structure. Soundness rests on a frame property that Rust’s ownership discipline provides: a call can affect only what it is passed.

1.3 Why this is not impossible

Pédrot and Tabareau’s fire triangle [18] shows that a type theory cannot have substitution of arbitrary terms, dependent elimination and observable effects at once without becoming inconsistent. Ochr is call-by-value, keeps dependent elimination, and restricts what may be substituted into a type: a type only ever contains values. A computation mentioned in a type is run on a private copy of the environment, and only its observation, a value, enters the type. Ownership is what makes this restriction cheap. A closed Ochr program can mutate only places it created, so every closed program behaves like a value, and the observation of an open program is a value of a known shape.

1.4 Contributions

- A core calculus, *Ochr*, combining mutable borrows with a dependent type theory with a universe of proof-irrelevant propositions, whose definitional equality unfolds imperative code (Section 3, Section 4).
- *Closing off*, which gives every stuck effectful call a neutral form made of sealed source programs, including calls that return borrows (Section 4).
- An observational equality between computations that is derived rather than primitive (Section 5).
- Worked examples, among them the equivalence of **AddM** with a version that first obtains a borrow of the final node and then writes through it, each proved by bare structural recursion (Section 2).
- A translation into the calculus of inductive constructions, in which the backward functions reappear, establishing consistency, and an adequacy theorem relating symbolic and concrete evaluation (Section 7).
- An executable implementation of the checker in Lean 4 that checks every example in the paper (Section 8).

2 Ochr by example

This section introduces *Ochr* through its running examples. Everything is informal here; Section 3 onwards makes it precise.

2.1 Imperative code is evaluated inside types

Ochr programs are made of definitions with parameters and a body. Borrow types $\&T$ are Rust’s mutable references; places are variables x , dereferences $*p$ and the predecessor field $p.1$ of a number. A **match** on a place binds its pattern variable to a *sub-place*, not a copy, so in **match** $*x$ { $Z \Rightarrow \dots$ | $S \ p \Rightarrow \dots$ } the name p stands for the field $(*x).1$ and $\&p$ reborrows it.

Add wraps **AddM** behind a pure interface. It owns its first argument, lends it to **AddM**, and returns it:

```
Add(x : Nat, y : Nat) : Nat := AddM(&x, y); x
```

Because definitional equality is evaluation, **Id** $\text{Nat } (\text{Add}(2, 3)) \ 5$ holds by **refl**: the checker runs **Add**(2, 3), which runs **AddM** in place on a local copy, and compares the result with 5. There is nothing special about imperative code here, and nothing special about **Add** being “pure”: it is simply a program whose observable output is its result.

The machine that does this is the symbolic semantics of Aeneas’s low-level borrow calculus [9]. An environment Ω maps variables to values; borrowing x moves its content into the borrow and leaves a *loan* behind; ending the borrow moves the content back. Running **Add**(2, 3) passes through these states:

```
{ x ↦ 2 }                                // enter Add
{ x ↦ loan₀ } ⊢ AddM(borrow₀ 2, 3)        // &x: the content moves into the borrow
{ x ↦ loan₀ | x' ↦ borrow₀ (S loan₁), ... } // AddM matched S and reborrowed the field
...
{ x ↦ 5 }                                // borrows ended, contents returned
```

2.2 Symbolic inputs, and what a stuck call leaves behind

To reason about all numbers at once, the checker runs programs on *abstract values* σ , which play the role of Lean’s free variables. Running **Add**(σ , 0) borrows $x \mapsto \sigma$ and calls **AddM**(**borrow**₀ σ , 0), whose body immediately gets stuck: it must match on σ .

In a pure type theory a stuck term is its own normal form. Here the call has also produced something that lives in the environment, namely the final content of the place it borrowed, and that content needs a name. We *close off* the call: the call returns, and the loan it held is filled with a *sealed program*, a closed source program that owns its state and computes that content:

```
{ x ↦ loan₀ } ⊢ AddM(borrow₀ σ, 0)
→ { x ↦ [let c = σ; AddM(&c, 0); c] } ⊢ ()
```

Write $N(\sigma)$ for $[let\ c = \sigma; AddM(\&c, 0); c]$. It is a neutral value, like a stuck `Nat.rec` in Lean, and it is literally the body of `Add(σ, 0)`: the sealed program *is* the pure wrapper, generated on demand. It is also exactly the value that Aeneas’s backward function for `AddM` would compute [9], written without leaving the source language. When a later case split refines σ to $S\ \sigma'$, $N(S\ \sigma')$ runs again, now makes progress through one successor, closes off the inner recursive call, and normalises to $S\ N(\sigma')$.

2.3 Equality between computations

The type `Id Unit (AddM(x, 0)) ()` compares two computations. It is evaluated at the environment where it is formed. Each side runs on its own copy of that environment; the result and the final contents of the places the side may write form its *observation*; and `Id` computes to ordinary equality between the two observations. For a definition taking $x : \&Nat$, the checker works at the definition’s *generic call* `AddMZero(&c)` from $\{ c \mapsto \sigma \}$, so the place that may be written is c , the owner of x ’s loan:

```
[[AddM(x, 0)]] = ((), N(σ))      // the call is stuck, so c receives the sealed program
[()]          = ((), σ)         // nothing happens; ending x's borrow returns σ to c

Id Unit (AddM(x, 0)) () ≡ Eq (Unit × Nat) ((), N(σ)) ((), σ) ≡ Eq Nat N(σ) σ
```

The last step uses the observational computation rules for equality [4]: equality of pairs is a conjunction, reflexive equations are τ , and τ is a unit for $\wedge$. The statement “running `AddM(x, 0)` has no effect” has become the ordinary proposition that $N(\sigma)$ equals σ .

2.4 The environment performs the congruence

Now the proof:

```
AddMZero(x : &Nat) : Id Unit (AddM(x, 0)) () :=
  match *x { Z => refl | S p => AddMZero(&p) }
```

The match on $*x$ splits on σ . In the Z branch the goal is `Eq Nat N(Z) Z`; $N(Z)$ runs to Z , the equation is reflexive and so τ , and `refl` proves it. In the S branch the goal is `Eq Nat N(S σ') (S σ')`, which normalises to `Eq Nat (S N(σ')) (S σ')`.

The recursive call is checked by the same rule that produced the goal: the callee’s statement is evaluated at the call site, with its parameter bound to the argument. At the call site the environment is

```
{ c ↦ loan₀ | x ↦ borrow₀ (S loan₁) } and the argument is borrow₁ σ'
```

because matching $S\ p$ through the borrow and taking $\&p$ leaves the successor in place, around a loan for the predecessor. The owner of the argument’s loan is found by following it outwards: $loan_1$ sits inside x ’s borrow, whose loan sits in c . So the recursive call’s statement observes c , and running `AddM` on the argument fills $loan_1$ with $N(\sigma')$, after which c holds $S\ N(\sigma')$:

```
[[AddM(x', 0)]] at the call site = ((), S N(σ'))
[()] at the call site = ((), S σ')
AddMZero(&p) : Eq Nat (S N(σ')) (S σ')
```

This is the goal, symbol for symbol. The successor that a pure proof would add with `cong S` was supplied by the environment. The recursive call is a proof, and proofs are erased at runtime, so the checker does not run it: its borrow argument is returned unchanged.

For comparison, here is the same theorem about the pure wrapper:

```
AddZero(x : Nat) : Id Nat (Add(x, 0)) x :=
  match x { Z => refl | S p => cong S (AddZero(p)) }
```

$\text{Add}(x, 0)$ writes nothing outside itself, so its observation is just its result, and the goal is again $\text{Eq Nat } N(\sigma) \sigma$, the very proposition AddMZero 's statement computed to. The induction hypothesis $\text{AddZero}(p)$ is about a fresh copy of the predecessor, so the successor must be added by hand. In Ochr the in-place proof is the shorter one. The two theorems are also interchangeable: $\text{AddZero}(x) := \text{AddMZero}(\&x)$ type-checks, because both statements normalise to the same proposition.

2.5 Returning a borrow

A more idiomatic in-place addition first finds the final node and then writes through it:

```
TailM(x : &Nat) : &Nat := match *x { Z => x | S p => TailM(&p) }
AddM'(x : &Nat, y : Nat) : Unit := let t = TailM(x); *t := y
```

TailM returns a borrow into its argument. When it is stuck on σ , the place it borrowed cannot be given its final content yet, because that depends on what the caller will later write through the returned borrow. Closing off such a call returns a borrow of a fresh k and fills the argument's loan with a sealed program that has a *hole* loan_k for the returned borrow's final content:

```
{ c ↦ loank } ⊢ TailM(borrowk σ)
  → { c ↦ [let c = σ; let r = TailM(&c); *r := loank; c] } ⊢ borrowk [let c = σ; let r
    = TailM(&c); *r]
```

In AddM' , the caller writes y through t and drops it; ending borrow_k substitutes the final content y for the hole. What remains in c is $[\text{let } c = \sigma; \text{let } r = \text{TailM}(\&c); *r := y; c]$: the effect of AddM' , as a program. The hole is the neutral form of an Aeneas region abstraction, and of a RustHorn prophecy [15]: it records only the *final* value written through the returned borrow.

The equivalence of the two additions is proved by the same bare recursion:

```
AddMEq(x : &Nat, y : Nat) : Id Unit (AddM(x, y)) (AddM'(x, y)) :=
  match *x { Z => refl | S p => AddMEq(&p, y) }
```

In the successor branch, re-running AddM' 's sealed program on $S \sigma'$ unfolds TailM once, closes off its inner call with a fresh hole, and the pending write $*r := y$ then fills that hole; on the other side the recursive call's statement, evaluated at the call site, performs the same steps through the caller's borrow. Both sides arrive at $\text{Eq Nat } (S A(\sigma', y)) (S B(\sigma', y))$, where A and B are the sealed programs for the two additions on the predecessor. The theorem for an owned number follows by lending it: $\text{AddMEqOwned}(x : \text{Nat}) : \text{Id Unit } (\text{AddM}(\&x, 0)) (\text{AddM}'(\&x, 0)) := \text{AddMEq}(\&x, 0)$.

2.6 Branching

A `match` on an abstract value in the middle of a function cannot pick an arm. Each arm is checked separately, and the rest of the function is then checked once, from the state in which the match itself has been closed off like a call. This covers borrows whose origin depends on the branch:

```
AddToOne(b : Nat, x1 : &Nat, x2 : &Nat, y : Nat) : Unit :=
  let r = match b { Z => x1 | S _ => x2 }; AddM(r, y)
```

The closed-off match returns a borrow with a hole that appears in the sealed programs for both x_1 's and x_2 's places; whichever the match would have chosen receives the final content. A proof about AddToOne splits on b , after which the sealed programs run and the goal becomes a statement about AddM alone.

3 The calculus

Ochr is a dependent type theory in the style of Lean’s kernel [6], extended with places, mutable borrows and sequencing. This section gives its syntax and the runtime structures of its evaluator. We keep the core small: the only inductive types are natural numbers and the unit type, plus pairs and the propositional connectives needed by equality. Section 10 discusses what generalises directly and what does not.

3.1 Syntax

```

terms  t, u, A, B ::= x | Prop | Typei
                    |  $\Pi(x_1:A_1 \dots x_n:A_n). B$                 dependent function type
                    | fix f (x1:A1 ... xn:An) : B by xj := t  function (recursive on xj, or
not at all)
                    | t(u1, ..., un)                        saturated call
                    | Nat | Z | S t | Unit | () | A × B | (t, u) | t.1 | t.2
                    | Eq A t u | refl | J(A, a, b, P, h, t) |  $\tau$  | P ∧ Q | (h, k)
                    | &A                                       borrow type
                    | p | &p | p := t | let x = t; u | t; u
                    | match p { Z => t | S y => u }
                    | Id A t u                                equality of computations
places p ::= x | *p | p.1

```

Figure 1: Syntax of Ochr. Eq, τ and $\wedge$ live in **Prop**. A fix without by is an ordinary λ .

Figure 1 gives the syntax. Types and terms share one grammar, as in any pure type system. Functions are n-ary and calls are saturated: a partial application would be a closure capturing its arguments, and a closure capturing a borrow is outside the core. A recursive function names the parameter it recurses on.

The imperative fragment is small. A *place* p is a variable, a dereference $*p$, or the predecessor field $p.1$ of a number. A place used as a term reads it; $\&p$ borrows it; $p := t$ assigns it; $\text{let } x = t; u$ introduces a new place x . The pattern variable of a *match* is a *sub-place*: in $\text{match } p \{ S y \Rightarrow u \}$, y stands for $p.1$, and nothing is copied.

$\&A$ is the type of a mutable borrow of an A . In the core, borrow types occur only as the types of variables, parameters and results, never inside another type: there are no borrows stored in data structures and no borrows of borrows. There are no shared borrows and no loops. Recursion is structural.

The propositional fragment is Lean’s: **Prop** is an impredicative universe with definitional proof irrelevance, erased at runtime, and **Eq** is its equality. $\text{Id } A \ t \ u$ compares two *computations* t and u of type A ; Section 5 shows that it is not a new primitive.

3.2 Values and environments

values	$v, w ::= Z \mid S \ v \mid () \mid (v, w) \mid * \mid \text{closures} \mid \text{types}$
	$\mid \text{borrow}_\ell \ v \mid \text{loan}_\ell \mid \perp \mid n$
neutrals	$n ::= \sigma \mid [t]$
environments	$\Omega ::= \text{a stack of frames of bindings } x : A \mapsto v$

Figure 2: Runtime structures.

The evaluator manipulates the values of Figure 2. Following the low-level borrow calculus of Aeneas [9], a borrow carries the borrowed content with it: $\text{borrow}_\ell \ v$ is a borrow, identified by ℓ , whose current content is v , and loan_ℓ is the placeholder left at the place it was taken from. When the borrow ends, its content is put back where the loan is. We depart from Aeneas in one respect: **a loan is a variable bound by its borrow**. It may occur anywhere inside a value, including inside a sealed program, and inside sealed programs it may occur more than once; ending the borrow substitutes its content for every occurrence. $\perp$ marks a place whose content has been moved out, and $*$ is the value of a proof.

Neutral values are stuck computations. An *abstract value* σ is an unknown value of a known type, playing the role of a free variable in Lean’s kernel: the checker introduces one for every parameter of a definition it checks, and nowhere else. A *sealed program* $[t]$ is a closed program whose run is stuck on an abstract value. Sealed programs are produced by closing off stuck calls (Section 4) and are the only neutral form besides abstract values. A value is *borrow-free* if it contains no borrow and no live loan.

An environment Ω is a stack of frames, one per active call, each binding variables to values. $\text{content}(\Omega, p)$ follows a place through variables, borrows ($*p$ looks inside $\text{borrow}_\ell v$ at v) and successors ($p.1$ looks inside S v at v). We write $\Omega[p \mapsto v]$ for the environment with the content of p replaced.

4 Evaluation and closing off

Definitional equality in Ochr is normalisation by one deterministic machine. With effects, rewriting-anywhere is not confluent: evaluating $(\lambda y. (x := 2; y))(x := 1)$ argument-first and substituting-first leave x holding different numbers. A fixed call-by-value order restores uniqueness, and the machine then plays the role that normalisation by evaluation plays for pure type theories [1]: two terms are definitionally equal when the machine sends them to the same normal form. On concrete inputs it is an ordinary interpreter. On abstract inputs it runs until it has to inspect an abstract value, and the question the rest of this section answers is what it returns then. The judgement is big-step: $\langle \Omega, t \rangle \Downarrow \langle \Omega', v \rangle$ runs t from environment Ω to value v and environment Ω' , and a run may instead be *stuck* (it needs to inspect a neutral) or fail with a borrow error.

4.1 Borrows

$$\begin{array}{c}
\frac{\text{borrow}_\ell v \text{ in } \Omega}{\Omega \rightsquigarrow \Omega[\text{borrow}_\ell v := \perp][v/\text{loan}_\ell]} \text{ END} \quad \frac{v = \text{content}(\Omega, p)}{\langle \Omega, \&p \rangle \Downarrow \langle \Omega[p \mapsto \text{loan}_\ell], \text{borrow}_\ell v \rangle} \text{ BORROW} \\
\\
\frac{v = \text{content}(\Omega, p) \text{ borrow-free}}{\langle \Omega, p \rangle \Downarrow \langle \Omega, v \rangle} \text{ READ} \\
\\
\frac{\text{content}(\Omega, p) = \text{borrow}_\ell v}{\langle \Omega, p \rangle \Downarrow \langle \Omega[p \mapsto \perp], \text{borrow}_\ell v \rangle} \text{ MOVE} \quad \frac{\langle \Omega, t \rangle \Downarrow \langle \Omega_1, v \rangle \quad \Omega_2 = \text{drop}(\Omega_1, \text{content}(\Omega_1, p))}{\langle \Omega, p := t \rangle \Downarrow \langle \Omega_2[p \mapsto v], () \rangle} \text{ ASSIGN}
\end{array}$$

Figure 3: Borrows. Every rule that accesses a place p is preceded by [Access]: end every borrow whose loan lies on the path to p or inside its content. **drop** ends the borrows inside a dropped value and fails on a live loan in a dropped owned value.

Figure 3 gives the rules for borrows. They are Aeneas’s, simplified by treating loans as variables.

Ending a borrow [End] replaces it by $\perp$ and substitutes its content for its loan. There is no side condition: if the content itself contains loans (because something has reborrowed part of it), they travel with it, and the reborrows remain valid. Ending is allowed at any time; the machine does it lazily, when a place is accessed. [Access] makes every access exclusive: before a place is read, borrowed, assigned or matched on, every borrow whose loan lies on the path to it or inside its content is ended. A borrower that has been ended holds $\perp$, and any later use of it is an error. This is the whole of the borrow checker: exclusivity of mutable borrows is not a separate analysis but a consequence of running the program.

Reading a place copies its content if the content is borrow-free [Read] and moves it out otherwise [Move]. Data is therefore copied, and borrows, which must stay unique, are moved; reborrowing is explicit ($\&*x$). A runtime system may implement a last-use copy as a move; an affine usage discipline outside the core licenses this, so that **AddM** performs no allocation. Inside types copying is essential: **Id Nat (Add(x, x)) x** must at least be a well-formed statement, and so must $x + x = 2 \cdot x$.

4.2 Calls and matches

A call evaluates its arguments left to right, each into a temporary of the current frame, so that [Access] can see and end them: $f(\&x, \&x)$ ends the first borrow when the second is taken, and the call then fails reading $\perp$. It

then pushes a frame binding the parameters, runs the body, and pops the frame, dropping its bindings. Popping a frame that still lends out one of its owned places is an error: something would outlive the place it borrows.

Types and proofs are erased at runtime, and the machine treats them accordingly: a term whose type is a proposition, or which is a type, is evaluated on a *private copy* of the environment, argument evaluation included, and the copy is then discarded. A proof may use mutation freely on its copy, but it cannot affect the real environment, so running a proof and skipping it are indistinguishable and the machine skips it, returning $\star$. A call whose head is itself a neutral (an abstract function, or a sealed program) is stuck at once.

`match p { Z => t | S y => u }` inspects the head of `p`'s content. `Z` selects the first arm, and `S v` selects the second with `y` standing for the sub-place `p.1`. Through a borrow, `match *x` leaves the successor in the borrowed content, so `&y` then reborrows the predecessor field in place: $x \mapsto \text{borrow}_0 (S \text{ loan}_1)$ and the new borrow is `borrow1 v`. If the head is a neutral, the run is stuck.

4.3 Closing off

When the body of a call is stuck, the call itself is stuck, and so is everything waiting for it. A pure type theory would return the stuck call as a neutral term. An effectful call cannot simply be returned: part of what it produces is the final content of the places it borrowed, and the caller's environment needs those contents before it can continue. We *close off* the call instead.

the body of $f(w_1, \dots, w_n)$ is stuck, $w_i = \text{borrow}_{\ell_i} u_i$ for $i \in I$, u_i loan-free

$$L := \text{let } c_i = u_i \quad (i \in I) \quad C := f(a_1, \dots, a_n), \quad a_i = \begin{cases} \&c_i & \text{if } i \in I \\ w_i & \text{otherwise} \end{cases}$$

result type	result	each loan_{ℓ_i} becomes
Unit	<code>()</code>	$[L; C; c_i]$
borrow-free D	$[L; C]$	$[L; C; c_i]$
$\&T$	$\text{borrow}_k [L; \text{let } r = C; *r]$	$[L; \text{let } r = C; *r := \text{loan}_k; c_i]$

Figure 4: [Close]. The partial run of the body is discarded; the borrows `borrowℓi` are consumed; `k` is fresh.

Figure 4 gives the rule. Take the call at the point where its arguments have been evaluated. For each borrow argument `borrowℓi ui`, the canonical program `L; C` recreates the situation the call was in, but with the borrowed contents owned by fresh local places `ci`: it moves each `ui` into `ci` and calls `f` on `&ci`. This program is closed. Running it and reading `ci` afterwards computes exactly what the call would have left behind the borrow, so each loan is filled with the sealed program $[L; C; c_i]$. The call returns the sealed program $[L; C]$, or `()` if its result type is **Unit**.

Sealed programs are the backward functions of Aeneas, written in the source language. For `AddM(borrow0 σ, y)`, the loan receives $[\text{let } c = \sigma; \text{AddM}(\&c, y); c]$, which is the body of `Add(σ, y)`. Where Aeneas's translation would emit `add_m_back σ y`, Ochr writes the program that defines it. Nothing about the call is lost: when `σ` is later refined, the sealed program runs again.

Returning a borrow. If `f` returns a borrow, the contents of the places it borrowed are not determined when the call returns: they depend on what the caller will write through the returned borrow before it ends. The rule therefore returns a borrow of a fresh `k`, whose content is the sealed program for the returned borrow's initial content, and fills each `loanℓi` with a sealed program that has a *hole*: it writes `loank` through the returned borrow before reading `ci`. Because loans are variables, the hole is filled by the ordinary [End] rule: when the caller drops the returned borrow with final content `w`, `w` is substituted for `loank`. This records only the *final* value written through the returned borrow, which is all that the backward function of Aeneas takes as input, and the hole is the neutral form of an Aeneas region abstraction and of a RustHorn prophecy [15]. If `f` has several borrow arguments, the hole occurs in each of their sealed programs, and one [End] fills them all.

The precondition that each `ui` is loan-free is guaranteed by [Access], which ended those loans when the argument was read or borrowed. Without it, a live loan would be copied into a sealed program and survive the borrow that binds it.

4.4 Sealed programs and refinement

Values are kept in normal form. The normal form of a sealed program $[L; C; K]$ is obtained by running it from the empty environment, with one restriction: its *head call* C may not itself be closed off (calls made inside C 's body may). This is the analogue of the guard on fixpoint unfolding in the calculus of inductive constructions. Without it, a sealed program would re-close into itself forever; with it, a sealed program either runs to a value or stays exactly as it is. A loan whose borrow lives outside the run (a hole whose returned borrow is still live) is *inert*: the run treats it like an abstract value.

A *refinement* substitutes a constructor pattern for an abstract value, $\sigma := Z$ or $\sigma := S \sigma'$, when a case split learns it (Section 6). Substitution into a sealed program is followed by normalisation, which runs the sealed program again. For $N(\sigma) = [\text{let } c = \sigma; \text{AddM}(\&c, \emptyset); c]$ and $\sigma := S \sigma'$, the head call **AddM** now unfolds, matches the successor, reborrows the predecessor, and makes the inner recursive call, which is stuck on σ' and is closed off; so $N(S \sigma') = S N(\sigma')$. Normal forms of this shape are what makes the induction of Section 2 go through. The key property is that refinement commutes with closing off: refining the arguments of a closed-off call and re-normalising gives the same normal form as closing off the call on the refined arguments (Section 7).

Stuck blocks. A stuck **match** that is not the body of a call is closed off in the same way, as the body of a call to an anonymous function of its free places. The function takes its free places the way Rust infers closure captures: a place the match moves out of, in any arm, is moved in; a place it writes or borrows is passed by borrow; a place it only reads is copied. This covers stuck matches inside types and, during checking, a stuck match that is followed by more code (Section 6).

5 Observational equality

Id A t u states that the computations t and u , both of result type A , are indistinguishable from where the statement is made: run from the current environment, they return equal values and leave equal contents in every place either of them may write. This section defines that notion precisely and shows that it reduces to ordinary propositional equality.

5.1 Footprints and observations

A computation affects the world only through the places it names. The places a term may **write** are read off its syntax: they are the free places that appear under $\&_$ or on the left of $:=$, and the borrow variables it mentions, since a borrow grants write access to someone else's place. Each of these is traced back to the owned place that will receive the write. For a place rooted at an owned variable x , that is x . For a borrow variable holding **borrow _{ℓ}** v , it is the set of *owners* of ℓ :

$$\text{owners}(\ell) = \bigcup_{\text{occurrences of loan}_\ell} \begin{cases} \text{owners}(m) & \text{if the occurrence lies inside the content of borrow}_m \\ \{x\} & \text{if it lies inside the owned binding } x \end{cases}$$

Following loans outward through enclosing borrows is what connects a reborrow with the place it ultimately writes. In the successor case of **AddMZero**, the argument **borrow₁** σ' has its loan inside the content of x 's **borrow₀**, whose loan is in the generic caller's c ; so the owner of the recursive call's argument is c , the same place the goal observes. The owners form a set because a hole left by a returned borrow can occur in several sealed programs at once. Observing only one of them would be unsound: a call that returns a borrow into one of two arguments could then be proved to leave the other unchanged whichever it picked.

The *footprint* $W(t, u)$ is the union of the owners of the places t or u may write. It is syntactic, so it does not change when a later case split refines an abstract value, and it does not depend on unrelated places in the environment.

The *observation* of t at Ω on footprint W runs t on a private copy of Ω , ends every borrow that remains, and reads off the result and the contents of the footprint:

$$\llbracket t \rrbracket_\Omega^W = (v, \Omega'(k)_{k \in W}) \quad \text{where } \langle \Omega, t \rangle \Downarrow \langle \Omega'', v \rangle \text{ and } \Omega'' \overset{*}{\rightsquigarrow} \Omega' \text{ with no borrows left}$$

If $\mathfrak{t}$ is itself stuck outside any call, it is first closed off as a stuck block, so an observation always exists. Ending every remaining borrow makes the observation independent of which borrows happened to be live when $\mathfrak{t}$ finished; that the order in which they are ended does not matter is a theorem (Section 7).

5.2 Id is equality of observations

$$\text{Id } A \ t \ u \equiv \text{Eq } (A \times T_W) \llbracket t \rrbracket_{\Omega}^W \llbracket u \rrbracket_{\Omega}^W \quad W = W(t, u), \text{ both sides run from } \Omega \text{ on independent copies}$$

$$\text{Eq } (A \times B)(a, b)(a', b') \equiv \text{Eq } A \ a \ a' \wedge \text{Eq } B \ b \ b' \quad \text{Eq } A \ a \ b \equiv \top \quad \text{if } a \equiv b \quad \top \wedge P \equiv P \equiv P \wedge \top$$

Figure 5: **Id** computes to **Eq** between observations (T_W is the product of the types of the places in W ; for empty W the pair is just A), and **Eq** computes on pairs and on reflexive instances. **refl** : $\top$.

Figure 5 is the whole of **Id**. Both sides run from the same environment, each on its own copy; sharing one copy and running the sides in sequence would let the first side’s effects change what the second observes. The result type A must be borrow-free: a borrow has no meaning once the computation that produced it has finished.

The equality **Eq** on the right is Lean’s, living in the proof-irrelevant universe **Prop**. We add three conversion rules in the style of observational type theory [4, 17]: equality at a product is a conjunction, a reflexive equation is $\top$, and $\top$ is a unit for $\wedge$. Their only role is to strip away the parts of an observation that no side changed, so that **Id Unit (AddM(x, 0)) ()** computes to the single interesting equation

$$\text{Eq } (\text{Unit} \times \text{Nat})((), N(\sigma))(\(), \sigma) \equiv \top \wedge \text{Eq Nat } N(\sigma) \ \sigma \equiv \text{Eq Nat } N(\sigma) \ \sigma.$$

Each rule identifies two propositions with the same truth value, so all three hold in the proof-irrelevant set model of Lean’s type theory [6]. Accordingly **refl** proves $\top$, and by conversion every reflexive equation. Transport **J**(A , a , b , P , h , $\mathfrak{t}$) takes its endpoints explicitly, since a reflexive **Eq** $A \ a \ a$ computes to $\top$ and no longer records them.

Three consequences are worth drawing out.

Effects are outputs. The observation turns every write to a place in the footprint into an extra component of the result. This is the sense in which **Ochr** has no pure/impure divide: a computation’s effects are part of its value as seen by **Id**, and the programmer never declares them.

Owned locals are observed. The footprint includes places the current function owns, not only the places behind its borrow parameters. This is forced. A context such as $(\square; x)$, which reads x afterwards, distinguishes $x := 6$ from $()$; if **Id** ignored owned locals it would equate them, and transport along that equation would prove **Eq Nat** 6 5.

Unobserved moves are harmless. Reading a borrow variable moves it, so the two sides of an **Id** may consume different borrow variables. Borrow variables are never in the footprint (their owners are), and ending the remaining borrows puts every borrowed content back in its owner, so what one side consumed and the other did not makes no difference to the comparison.

6 Typing

Typing is the machine run on abstract inputs, with case splitting where the checked program itself inspects an abstract value. The judgement $\Omega \vdash \mathfrak{t} \Downarrow v : A \dashv \Omega'$ says that from Ω the term $\mathfrak{t}$ runs to the value v of type A , leaving Ω' . It returns the value, not just the type, because later types may depend on it: after **let** $z = (\text{let } y = \&x; *y := 2; x)$, the type **Id Nat** $z \ 2$ holds by **refl** only because the checker knows that z is 2. The rules for the basic forms are the evident ones, each the corresponding machine rule annotated with types, and we omit them. Figure 6 gives the rules that are particular to **Ochr**.

[**Call-type**] At the point where the arguments of $f(\bar{a})$ have been evaluated, with $f : \Pi(\bar{x} : \bar{A}) . B$: push a frame binding each x_i to a_i 's value, evaluate B there on a private copy of the environment, and pop. The result is the type of the call. Each a_i must have type A_i , evaluated with the earlier parameters bound.

[**Def**] $\text{fix } f (\bar{x} : \bar{A}) : B \text{ by } x_j := b$ is checked at its *generic call*: from an environment with a fresh owned place $c_i \mapsto \sigma_i$ for each borrow parameter $x_i : \&T_i$, the call $f(\bar{a})$ with $a_i = \&c_i$ for borrow parameters and $a_i = \sigma_i$ otherwise. The goal is the [Call-type] of this call. The body runs in the pushed frame, which is then dropped; its result's type must be convertible to the goal, as refined by the splits on the way.

[**Split**] When the checked program matches on a place whose content has an abstract head σ , each arm is checked with the refinement $\sigma := Z$, respectively $\sigma := S \sigma'$ for fresh σ' , applied to the environment, the goal and every stored type. If the head is a sealed program, every occurrence of it is first replaced by a fresh σ . If the match is followed by more code, that code is checked once, from the state in which the match has been closed off as a stuck block.

[**Rec**] In the body of $\text{fix } f \dots \text{by } x_j$, f occurs only as the head of a call, and each recursive call passes in position j a value, or a borrow of a value, that is a strict subterm of x_j 's entry value σ_j as refined so far.

Figure 6: The typing rules particular to Ochr.

6.1 One rule for goals and induction hypotheses

A definition is checked by running it at its most general call [Def]. For $\text{AddMZero}(x : \&\text{Nat})$ that is $\text{AddMZero}(\&c)$ from $\{ c \mapsto \sigma \}$: the parameter becomes a borrow of a fresh owned place, which stands for whatever the real caller lends. The goal is then computed by [Call-type], the rule that types every call.

The same rule types the recursive call in the successor case, at a different environment. There the argument $\&p$ borrows the predecessor field of c 's content, so evaluating the callee's statement at the call site observes c through the successor that surrounds the field. This is how the induction hypothesis arrives already wrapped in S . The generic call and the recursive call are two instances of one rule, which is why the goal and the hypothesis meet without a congruence step. That the callee's statement, proved at its generic call, remains true at every call site is the frame property of Section 7: a call affects only what it is passed, so the caller's observation is the callee's, placed in a context.

6.2 Types are formed once

A type is evaluated once, when it is formed, on a private copy of the environment, and becomes a closed statement about values. Later mutation cannot change it. This includes the codomain of a Π -type, which is a closure over the values of the variables it mentions; only its parameters are bound later, at each call. The goal of a definition is formed at its generic call, before the body runs, so a body that mutates its argument and then proves something about the new contents does not prove the goal: $\lambda(x : \&\text{Nat}). (*x := 5; \text{refl})$ does not have type $\Pi(x : \&\text{Nat}). \text{Id Nat } (*x) 5$.

6.3 Case splitting and joins

A *match* in the checked program on an abstract value cannot pick an arm, so each arm is checked with the abstract value refined [Split]. What follows the match is checked once, from a state in which the match itself has been closed off as a stuck block. The closed-off match is precise: its sealed programs contain the match, and they reduce as soon as a later split decides it. There is no join of environments and no loss of information, and a borrow whose origin depends on the branch is handled like any other returned borrow. AddToOne of Section 2 is accepted, and so is any function whose body is split into a helper at a match: a match and a call to a function containing it are treated alike. Errors in the code after a match are reported once, at the line where they occur, not once per arm.

6.4 Erased terms leave no trace

Types and proofs are erased at runtime. The machine mirrors this exactly: it evaluates a type, or a term whose type is a proposition, on a private copy of the environment, argument evaluation included, and discards the copy. Two things follow. A formed type is a closed statement about values, as described above. And a proof can have

no effect on the program around it: it may mutate places freely, but only on its own copy. Running a proof and skipping it are therefore indistinguishable, and the machine skips proofs, which is what the compiled program does too.

This is one principle, not a restriction on proofs. **AddMZero**’s successor case borrows the field **p** to pass it to the induction hypothesis; the borrow happens on the private copy, and the real environment is untouched. Nothing in a program is marked pure, and any program may appear in a statement.

6.5 Why each condition is there

Each side condition above was added in response to a concrete false proof or run-time error found while designing the calculus. We list them because together they delimit the design, and because each is a regression test for the implementation (Section 8).

- **Π -types capture values.** If the codomain of a hypothesis $h : \Pi(_ : \text{Unit}). \text{Id Nat } x \text{ Z}$ were re-evaluated at each call, then $x := S \ x$; $h()$ would produce a proof of $\text{Id Nat } (S \ Z) \ Z$ from h , which was proved when x was Z .
- **Recursion on entry values.** A syntactic structural check accepts $f(x : \&\text{Nat}) := *x := S \ *x$; $\text{match } *x \{ S \ p \Rightarrow f(\&p) \}$, whose recursive argument is the parameter’s original value, and with it a proof of $\Pi(n : \text{Nat}). \perp$. The recursive function must also never escape as a value, or the check can be bypassed through a higher-order call.
- **Erased terms run on a private copy.** Proof irrelevance at a Π -type over a borrow identifies $\lambda x. *$ with $\lambda x. (*x := 7; *)$; if calls to these could affect the caller, transport along that identification proves $\perp$. Stipulating only that proof *calls* are skipped is not enough: a proof block that writes, closed off after a split and then refined, would be skipped, but run directly it would not, and a closed proof of $\text{Id Nat } (S \ Z) \ Z$ results. Running every erased term on a private copy makes skipping a consequence rather than a stipulation, and makes it commute with refinement. For the same reason a type must be formed on a private copy: a type that writes would otherwise make the checker and the compiled program disagree.
- **All owners are observed.** A function returning a borrow into one of two arguments leaves its hole in both. A footprint that observed only one owner lets [Call-type] prove $\text{Id Nat } (S \ Z) \ Z$.
- **Access is exclusive and closing off needs loan-free arguments.** Moving a borrow whose content still holds a live loan into a call, and closing that call off, copies the loan into a sealed program; after refinement the sealed program disagrees with running the call, and the compiled program writes through an ended borrow.

7 Metatheory

Draft pending the model.

8 Implementation

Draft pending the Lean checker.

9 Related work

Functional translation of borrows. Aeneas [9] translates safe Rust into a pure functional program in Lean, F* or Coq. Its symbolic semantics of the low-level borrow calculus is the direct ancestor of our machine: we take its values with loans and borrows, its lazy ending of borrows, and its insight that a mutable borrow parameter is a forward computation plus a backward function. Later work gives that symbolic semantics joins and loops and proves it sound as a borrow checker [10]. The difference is where and when the translation happens. Aeneas translates eagerly, whole-program, once, and the user reasons about the translated artifact, where backward functions are named definitions. Ochr never produces a translated program. The same state-passing content appears lazily inside conversion, only as far as a particular equation needs, and only in source syntax: a sealed

program is what Aeneas would call an application of a backward function. Aeneas is modular in that it never unfolds a callee; Ochr must unfold, because unfolding is what definitional equality is, and closing off is what keeps the unfolding finite.

Prophecies. RustHorn [15], RustHornBelt [14] and Creusot [7] model a mutable borrow as its current value paired with a *prophecy* $\wedge x$ of its final value, resolved when the borrow ends. A filled loan in Ochr is a prophecy that has been resolved by the machine, and the hole in a sealed program for a returned borrow is an unresolved one. Prophecies are logic variables introduced at borrow creation; Ochr introduces no variables at all outside binders, which is what keeps concrete evaluation free of symbols (§Section 7).

Specification-based verification of imperative code. Verus [11], Creusot, Prusti, Flux [12] and VeriFast verify imperative code against specifications written separately, in pre- and postconditions, refinement types or separation logic, and discharge the obligations with automated provers. Hoare type theory [16] and Ynot internalise the triples in a dependent type theory; F*’s Dijkstra monads [2] internalise weakest preconditions and let **reify** turn a stateful computation into a state-passing function. In all of these the effectful program is specified, not compared: two programs with the same specification are not thereby equal. In Ochr the specification of an in-place function is another program, or a theorem about the function itself, and equality of computations is a type.

Effects in dependent type theory. The fire triangle [18] shows that substitution, dependent elimination and observable effects cannot coexist consistently, and ∂ CBPV organises the possible compromises. Ochr sits in the call-by-value corner: dependent elimination is kept, and substitution into types is restricted to values, where a computation in a type contributes only its observation. The observation of a computation is *thunkable* in their sense, and ownership gives a syntactic criterion for thunkability, which they note is otherwise hard to capture intensionally. Ahman’s eMLTT [3] equips computations with definitional equations from an algebraic theory of global state; Ochr’s state is structured, local and borrow-shaped, and its equations come from running the program rather than from an axiomatised theory.

Observational equality. Observational type theory [4, 17] computes the equality type by the structure of the type: equality of pairs is a pair of equalities, equality of functions is pointwise. Ochr’s **Id** between computations is an instance of this idea at the type of observations, and we adopt the observational computation rules for the data types of the core.

Pure languages with in-place updates. Lean 4 performs destructive updates on values it can prove unshared at runtime [19], and Koka’s fully in-place fragment [13] guarantees statically that a function allocates nothing. Programs in these languages are pure, so their proofs are ordinary, but the programmer writes functional code and cannot express a borrow that is returned to the caller. Linear and quantitative type theories [5] and ATS [20] track resources in types; Ochr uses the same separation between runtime usage and erased type-level usage as quantitative type theory, where a variable used in a type is not consumed.

Relational reasoning. ReLoC [8] and relational Hoare logics prove two programs equivalent inside a program logic. Ochr’s **Id** is a relational statement about two computations, but it is decided, where possible, by conversion, and it lives in the same type theory as the programs.

10 Discussion

10.1 What generalises directly

Inductive types. The core has only **Nat** and **Unit**, but nothing in the rules is specific to them. A constructor with several fields gives several sub-places **p.1**, ..., **p.k**; [Split] refines an abstract value to a constructor applied to fresh abstract values; [Rec] asks for a strict subterm of the entry value. In-place list append is **AddM** with **Cons** in place of **S**, and its theorems are proved the same way.

Opaque definitions. A definition may be declared with a type and no body, or used through an abstract function value. A call to it is stuck at once and closed off, so its effects are recorded as sealed programs mentioning it, and every theorem about its callers remains available. This is how Ochr recovers the modularity of Aeneas’s

symbolic execution [9], which never unfolds a callee: the programmer chooses, per definition, whether conversion may look inside.

Shared borrows. Immutable borrows are simpler than mutable ones in this setting: a shared borrow never has a final value different from its initial one, so closing off a call that takes one needs no sealed program for it. We omitted them to keep the core small.

10.2 The frontier

Borrows stored in data. The central restriction of the core is that borrow types never occur inside other types. It rules out an iterator that returns a list of borrows into a list, $\text{IterM} : \&(\text{List Nat}) \rightarrow \text{List } (\&\text{Nat})$. There, the number of borrows returned is itself unknown on symbolic input, so no fixed set of holes can stand for their final values. The natural generalisation is a hole that stands for a whole data structure of final values, filled when the last borrow in the structure ends, which is the neutral form of an Aeneas region abstraction over a symbolic value with a borrow projector. The same restriction excludes closures that capture borrows.

Loops. A loop is a tail-recursive local function, and its type is its invariant. Closing off a loop that is stuck on an abstract value is closing off that function. We have not designed the surface form, nor checked whether the invariants that arise are the ones programmers expect to write.

Interior mutability and unsafe code. Closing off is sound because a call can affect only what it is passed. Global state, interior mutability or unsafe aliasing would each break this frame property, and with it the rule. They would need to be confined, as Rust confines unsafe code behind safe interfaces, and given sealed-program semantics through their interfaces.

10.3 Costs

Checking cost. Type checking runs programs. On concrete data this is the cost of running them; on symbolic data each stuck call is closed off once, and each refinement re-runs the sealed programs that mention the refined value. Sealed programs are closed and canonical, so their normal forms can be cached and shared, and a refinement only re-runs what mentions it. Our implementation does no caching; **TODO: timings from the implementation**. We have not measured large programs.

Copying. The calculus copies borrow-free data on reads. A compiled program should move instead, and an affine usage discipline on runtime code, outside the core, licenses implementing each last-use copy as a move. Types are exempt from that discipline, which is what lets $x + x = 2 \cdot x$ be stated for a type without a copy operation; this is the same separation between runtime usage and erased usage as quantitative type theory [5].

What the programmer sees. Goals and hypotheses are propositions about values, and every value that appears in them is either data, an abstract value named after the parameter it came from, or a sealed program, which is source code. A sealed program can be printed as the user’s own wrapper when one exists ($\text{flet } c = \sigma; \text{AddM}(\&c, \theta); c$ is $\text{Add}(x, \theta)$). Nothing in a goal mentions loans, borrows, environments or backward functions. We regard this as a design invariant, and it is a useful test for extensions: a feature whose goals cannot be printed as source code is one the programmer would need a new concept to reason about.

11 Conclusion

The usual way to verify an efficient imperative program is to write it twice. We have shown that, for programs whose mutation is disciplined by Rust-style ownership, the second copy can be generated on demand, inside the type checker, as source code: when definitional equality meets a call it cannot run, it closes the call off into sealed programs that own their state, and a later case split runs them again. Combined with an observational equality that compares computations by their results and their writes, this lets theorems about in-place code be stated about that code and proved by the same structural recursion one would use for pure code. Often the in-place proof is the simpler one, because the borrow structure of the environment performs the congruence steps a pure proof must spell out.

The core calculus is small, and every side condition in it is there because a smaller one admitted a proof of false. The main open direction is borrows stored inside data, where the neutral form of a stuck call must describe a whole structure of final values rather than a fixed set of places.

Bibliography

- [1] Andreas Abel. 2013. Normalization by Evaluation: Dependent Types and Impredicativity. In *Habilitation thesis, LMU Munich*, 2013.
- [2] Danel Ahman, Catalin Hritcu, Kenji Maillard, Guido Martínez, Gordon Plotkin, Jonathan Protzenko, Aseem Rastogi, and Nikhil Swamy. 2017. Dijkstra Monads for Free. In *POPL*, 2017.
- [3] Danel Ahman. 2018. Handling Fibred Algebraic Effects. In *Proc. ACM Program. Lang. (POPL)*, 2018.
- [4] Thorsten Altenkirch, Conor McBride, and Wouter Swierstra. 2007. Observational Equality, Now!. In *PLPV*, 2007.
- [5] Robert Atkey. 2018. Syntax and Semantics of Quantitative Type Theory. In *LICS*, 2018.
- [6] Mario Carneiro. 2019. The Type Theory of Lean.
- [7] Xavier Denis, Jacques-Henri Jourdan, and Claude Marché. 2022. Creusot: A Foundry for the Deductive Verification of Rust Programs. In *ICFEM*, 2022.
- [8] Dan Frumin, Robbert Krebbers, and Lars Birkedal. 2018. ReLoC: A Mechanised Relational Logic for Fine-Grained Concurrency. In *LICS*, 2018.
- [9] Son Ho and Jonathan Protzenko. 2022. Aeneas: Rust Verification by Functional Translation. In *Proc. ACM Program. Lang. (ICFP)*, 2022. 116:1–116:31.
- [10] Son Ho, Aymeric Fromherz, and Jonathan Protzenko. 2024. Sound Borrow-Checking for Rust via Symbolic Semantics. In *Proc. ACM Program. Lang. (ICFP)*, 2024.
- [11] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. 2023. Verus: Verifying Rust Programs using Linear Ghost Types. In *Proc. ACM Program. Lang. (OOPSLA)*, 2023.
- [12] Nico Lehmann, Adam T. Geller, Niki Vazou, and Ranjit Jhala. 2023. Flux: Liquid Types for Rust. In *PLDI*, 2023.
- [13] Anton Lorenzen, Daan Leijen, and Wouter Swierstra. 2023. FP^2 : Fully in-Place Functional Programming. In *Proc. ACM Program. Lang. (ICFP)*, 2023.
- [14] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: A Semantic Foundation for Functional Verification of Rust Programs with Unsafe Code. In *PLDI*, 2022.
- [15] Yusuke Matsushita, Takeshi Tsukada, and Naoki Kobayashi. 2020. RustHorn: CHC-based Verification for Rust Programs. In *ESOP*, 2020.
- [16] Aleksandar Nanevski, Greg Morrisett, and Lars Birkedal. 2006. Polymorphism and Separation in Hoare Type Theory. In *ICFP*, 2006.
- [17] Loïc Pujet and Nicolas Tabareau. 2022. Observational Equality: Now for Good. *Proc. ACM Program. Lang. (POPL)* 6, (2022).
- [18] Pierre-Marie Pédro and Nicolas Tabareau. 2020. The Fire Triangle: How to Mix Substitution, Dependent Elimination, and Effects. *Proc. ACM Program. Lang. (POPL)* 4, (2020), 58:1–58:28.
- [19] Sebastian Ullrich and Leonardo de Moura. 2019. Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming. In *IFL*, 2019.
- [20] Hongwei Xi. 2003. Applied Type System (extended abstract). In *TYPES*, 2003.